

پیاده‌سازی TCP-Mini روی UDP/IP و تحلیل کنترل ازدحام

گزارش تمرین برنامه‌نویسی شماره ۴ — درس شبکه‌های کامپیوتری

دانشجو: سجاد تقی‌زاده

نیمسال: دوم سال تحصیلی ۱۴۰۴-۰۵

تاریخ گزارش: ۱۴۰۵/۰۴/۲۲ (2026-07-13)

این پروژه به‌صورت انفرادی انجام شده است.

۱. معماری کلی پروژه

پروژه شامل دو برنامه مستقل `sender.cpp` و `receiver.cpp` است که هر دو از یک `socket` از نوع `SOCK_DGRAM` (UDP) استفاده می‌کنند. تمام قابلیت‌های لایه انتقال قابل اعتماد — شماره‌گذاری بسته‌ها، ACK تجمعی، تایم‌اوت و ارسال مجدد، پنجره لغزان و کنترل ازدحام — به صورت کامل در سطح برنامه (application layer) پیاده‌سازی شده‌اند و هیچ کتابخانه آماده‌ای برای این بخش‌ها استفاده نشده است.

ساختار فایل‌ها دقیقاً مطابق پیشنهاد فاز صفر است:

- `packet.hpp` — قالب بسته، checksum، سریال‌سازی/دی‌سریال‌سازی
- `timer.hpp` — `Stopwatch` برای زمان‌سنجی و `RtoTimer` برای مدیریت تایمر ارسال مجدد
- `logger.hpp` — نویسنده `events.log` و `wnd.csv`
- `receiver.cpp` / `sender.cpp` — برنامه‌های اصلی فرستنده و گیرنده
- `Makefile` — کامپایل با `make`
- `scripts/run_experiments.py` — اجرای خودکار آزمایش‌های فاز پنج و رسم نمودارها

فرستنده به جای شماره‌گذاری بایتی، بسته‌ها را با **شماره ترتیبی segment** (packet index) شماره‌گذاری می‌کند که با مثال‌های صورت پروژه («اگر گیرنده تا بسته ۱۰ را دریافت کرده، ACK شماره ۱۱ را می‌فرستد») کاملاً هم‌خوان است و پیاده‌سازی و اشکال‌زدایی را ساده‌تر می‌کند.

پروتکل به شکل **Go-Back-N** با یک تایمر واحد برای قدیمی‌ترین بسته تأیید نشده عمل می‌کند: گیرنده فقط بسته‌ای را می‌پذیرد که دقیقاً شماره موردانتظار را داشته باشد؛ در غیر این صورت (بسته خارج از ترتیب یا تکراری) یک ACK تکراری برای همان شماره موردانتظار می‌فرستد. این رفتار هم مبنای منطق ACK تجمعی است و هم پیش‌نیاز طبیعی مکانیزم Fast Retransmit در فاز امتیازی.

برنامه با سه حالت قابل انتخاب (`--mode`) اجرا می‌شود که هرکدام معادل یکی از فازهای پروژه است:

mode	معادل فاز	توضیح
stopwait	فاز ۱	پنجره ارسال ثابت و برابر با ۱ بسته
window	فاز ۲	پنجره لغزان با اندازه ثابت (<code>--window-size</code>)
congestion	فاز ۴/۵	پنجره مؤثر برابر $\min(\text{receiver_window}, \text{cwnd})$ با Slow Start / Congestion Avoidance

شبیه‌سازی `loss` و `delay` (فاز ۳) در سطح گیرنده و مستقل از mode با پارامترهای `--loss` و `--delay` پیاده شده تا بتوان آن را روی هر سه حالت اعمال کرد.

۲. قالب Packet و فیلدهای Header

```
struct PacketHeader {
    uint32_t seq_num;    // (برای بسته‌های کنترلی خالص، صفر) شماره segment ترتیبی این
    uint32_t ack_num;    // (تجمعی ACK) بعدی مورد انتظار segment شماره
    uint16_t length;     // بر حسب بایت payload طول
    uint16_t checksum;   // checksum روی ۱۶ بیتی header + payload
    uint8_t flags;       // ترکیب DATA / ACK / FIN بیتی از
};
```

فیلد	کاربرد
seq_num	شماره بسته داده؛ گیرنده با مقایسه آن با expected_seq بسته‌های خارج از ترتیب یا تکراری را تشخیص می‌دهد.
ack_num	در بسته‌های ACK، شماره segment بعدی مورد انتظار گیرنده را اعلام می‌کند (رفتار تجمعی مشابه TCP واقعی).
length	طول payload برای بازسازی دقیق مرز بسته هنگام deserialize و اعتبارسنجی سازگاری اندازه بافر UDP.
checksum	تشخیص خطای بیتی؛ اگر checksum نامعتبر باشد بسته به صورت خاموش دور ریخته می‌شود (رفتاری معادل packet loss).
flags	نوع بسته را مشخص می‌کند: DATA، ACK، یا FIN (و ترکیب ACK FIN برای تأیید پایان انتقال).

تابع `computeChecksum` یک checksum مکمل یک ۱۶ بیتی به سبک IP/TCP روی `header` (با صفر فرض کردن فیلد checksum) به همراه `payload` محاسبه می‌کند. `serialize` فیلدهای عددی را با `htons / htonl` به ترتیب بایت شبکه تبدیل کرده و `header` را به بافر بایتی متصل به `payload` تبدیل می‌کند؛ `deserialize` عملیات معکوس را انجام داده و علاوه بر تطابق طول، خود checksum را نیز اعتبارسنجی می‌کند.

۳. نحوه کار Stop-and-Wait (فاز ۱)

در حالت `stopwait` اندازه پنجره ارسال همیشه برابر ۱ است: فرستنده یک بسته می‌فرستد، تایمر RTO (پیش‌فرض ۱۰۰۰ میلی‌ثانیه، قابل تنظیم با `--timeout-ms`) را شروع می‌کند و تا دریافت ACK یا انقضای تایمر، بسته بعدی ارسال نمی‌شود. اگر تایمر منقضی شود، همان بسته دوباره ارسال و تایمر بازنشانی می‌شود.

نمونه رویدادهای واقعی ثبت‌شده در `events.log` برای این حالت (فایل نمونه `sample_input.txt`، ۱۱۴۰۰۰ بایت، بدون loss):

```
[1.28ms] SEND seq=0 len=512
[1.62ms] ACK ack_num=1 cwnd=1.000000 ssthresh=16.000000
[1.66ms] SEND seq=1 len=512
[1.79ms] ACK ack_num=2 cwnd=1.000000 ssthresh=16.000000
[1.82ms] SEND seq=2 len=512
[1.90ms] ACK ack_num=3 cwnd=1.000000 ssthresh=16.000000
...
```

خروجی کنسول فرستنده و نتیجه مقایسه فایل خروجی با ورودی:

```
[sender] transfer complete
mode           : stop-and-wait
total packets   : 223
total bytes     : 114000
packets sent (w/ rtx) : 223
retransmissions : 0
timeouts       : 0
acks received   : 223
elapsed time (ms) : 18.29
throughput (bytes/s) : 6,233,400

$ diff sample_input.txt output_stopwait.txt
$ echo $?
0    فایل خروجی دقیقاً برابر فایل ورودی است -->
```

نتیجه: انتقال کامل و صحیح، بدون هیچ ارسال مجددی چون شبکه آزمایش (loopback) بدون افت بسته است.

۴. نحوه کار Sliding Window (فاز ۲)

در حالت `window`، فرستنده اجازه دارد تا `window_size` بسته متوالی را بدون انتظار برای ACK تک‌تک آن‌ها ارسال کند. گیرنده مانند قبل فقط بسته دقیقاً هم‌شماره با `expected_seq` را می‌پذیرد و ACK تجمعی (`ack_num = expected_seq` پس از پردازش) می‌فرستد. با دریافت هر ACK جدید (بزرگ‌تر از `send_base` فعلی)، فرستنده `send_base` را جلو می‌برد و در نتیجه پنجره «می‌لغزد» و بسته‌های جدید ارسال می‌شوند. تنها یک تایمر RTO برای قدیمی‌ترین بسته تأییدنشده فعال است؛ در صورت `timeout`، تمام بسته‌های بین `send_base` و `next_seq` دوباره ارسال می‌شوند (رفتار کلاسیک Go-Back-N).

```

[0.17ms] SEND seq=0 len=512
[0.22ms] SEND seq=1 len=512
[0.23ms] SEND seq=2 len=512
[0.23ms] SEND seq=3 len=512
[0.23ms] SEND seq=4 len=512
[0.24ms] SEND seq=5 len=512
[0.24ms] SEND seq=6 len=512
[0.25ms] SEND seq=7 len=512 <-- بسته پشتسرهم بدون انتظار برای ۸ (window_size=8)
[0.44ms] ACK ack_num=1 ...
[0.47ms] SEND seq=8 len=512 <-- پنجره جلو رفت

```

مقایسه Sliding Window و Stop-and-Wait

throughput (B/s)	elapsed (ms)	retransmissions	packets	window	حالت
6,233,400	18.29	0	223	1	Stop-and-Wait
31,831,400	3.58	0	223	8	Sliding Window

نتیجه: با $window=8$ زمان انتقال حدود ۵ برابر کاهش یافته است. علت این است که در Stop-and-Wait فقط یک بسته در هر RTT وارد شبکه می‌شود و ظرفیت لینک به‌طور کامل استفاده نمی‌شود؛ در Sliding Window چند بسته هم‌زمان «در پرواز» هستند و RTT بعدی همپوشانی می‌کند، بنابراین throughput مؤثر به‌جای $1/RTT$ به مقدار $window_size/RTT$ نزدیک می‌شود. (روی loopback محلی این تفاوت حتی محافظه‌کارانه دیده می‌شود؛ روی شبکه‌ای با RTT واقعی، بهبود سلینگ‌ویندو معمولاً چشمگیرتر است.)

۵. پیاده‌سازی Timeout و Retransmission

کلاس `RtoTimer` در `timer.hpp` یک تایمر ساده و تک برای قدیمی‌ترین بسته ارسال‌شده تأیید نشده (`send_base`) پیاده‌سازی می‌کند. حلقه اصلی فرستنده از `select()` برای انتظار هم‌زمان روی ورود ACK یا انقضای تایمر استفاده می‌کند تا CPU درگیر busy-wait نشود:

• هر بار که ACK جدیدی `send_base` را جلو می‌برد، تایمر بازنشانی (restart) می‌شود؛ اگر همه داده تأیید شده باشد، تایمر متوقف می‌شود.

• اگر تایمر منقضی شود، رویداد `TIMEOUT` ثبت شده و تمام بسته‌های بازه `[send_base, next_seq)` دوباره ارسال می‌شوند.

• در حالت `congestion`، هر `timeout` باعث `ssthresh = cwnd/2` و بازگشت `cwnd = 1` می‌شود.

مقدار RTO با `--timeout-ms` قابل تنظیم است (پیش‌فرض طبق صورت پروژه، ۱۰۰۰ میلی‌ثانیه).

۶. شبیه‌سازی Loss و Delay (فاز ۳)

گیرنده با پارامترهای `--loss P` و `--delay MS` اجرا می‌شود. برای هر بسته DATA دریافتی، یک عدد تصادفی `[0,1)` تولید می‌شود و اگر کوچک‌تر از `P` باشد، بسته بدون ارسال ACK دور ریخته می‌شود (دقیقاً معادل گم شدن بسته در شبکه). برای بسته‌های پذیرفته‌شده، قبل از ارسال ACK به اندازه `--delay` میلی‌ثانیه (`usleep`) تأخیر مصنوعی اعمال می‌شود. یک flag اختیاری `--seed` هم اضافه شده تا الگوی افت بسته در آزمایش‌های تکرارپذیر باشد (برای مقایسه منصفانه Fast Retransmit استفاده شده است).

نتایج آزمایش پنجره لغزان ثابت (`window=8`) روی `sample_input.txt` با مقادیر مختلف `loss`:

loss	delay	تطابق فایل	packets sent	retransmissions	timeouts	elapsed (ms)	throughput (B s)
0%	0ms	مطابق	223	0	0	3.83	29,774,200
5%	50ms	مطابق	295	72	9	14,226	8,013
10%	50ms	مطابق	345	122	16	16,499	6,909
20%	100ms	مطابق	6,971	6,748	848	279,859	407.3

نکته جالب: در سناریوی `loss=۲۰٪` با `window` ثابت و بدون کنترل ازدحام، فرستنده مجبور شد ۶,۹۷۱ بار بسته ارسال کند (برای فقط ۲۲۳ بسته منحصربه‌فرد!) و ۲۷۹.۸ ثانیه طول کشید. در همان درصد `loss` اما با `congestion --mode` (فاز ۵، آزمایش ۳ در بخش ۱۰)، با وجود فایل بزرگ‌تر (۵۸۶ بسته)، فقط ۱,۰۵۶ بسته ارسال و در ۱۰۴ ثانیه تمام شد. علت اصلی این است که پنجره ثابت (۸) هیچ‌گاه در برابر `loss` زیاد کوچک نمی‌شود؛ در نتیجه هر `timeout` کل پنجره ۸ تایی را از نو می‌فرستد و چون احتمال موفقیت کامل یک پنجره ۸ تایی زیر `loss=۲۰٪` فقط $0.8^8 \approx 17\%$ است، بیشتر تلاش‌ها دوباره `timeout` می‌خورند. کنترل ازدحام با `adaptime` با کوچک کردن `cwnd` به ۱ پس از هر ازدحام، احتمال موفقیت هر «دور» را بسیار بالاتر می‌برد و فرستنده تدریجی و مطمئن پیش می‌رود. این آزمایش به‌روشنی نشان می‌دهد چرا پنجره لغزان بدون کنترل ازدحام برای شبکه‌های واقعی و پرافت کافی نیست و چرا فاز ۴ پروژه (Slow Start / Congestion Avoidance) یک بهبود اساسی نسبت به فاز ۲ محسوب می‌شود.

چرا افزایش loss باعث افزایش retransmission و کاهش throughput می‌شود: در Go-Back-N، از دست رفتن یک بسته داده باعث می‌شود گیرنده تمام بسته‌های بعدی همان پنجره را (چون خارج از ترتیب هستند) دور بریزد و برای هر کدام یک ACK تکراری برای همان `expected_seq` بفرستد. فرستنده تا رسیدن timeout هیچ ACK جدیدی دریافت نمی‌کند و در نهایت **کل پنجره** را دوباره می‌فرستد، نه فقط بسته گم‌شده. بنابراین با افزایش احتمال loss:

1. تعداد timeout‌ها و به تبع آن retransmission‌ها به صورت غیرخطی افزایش می‌یابد (چون احتمال گم‌شدن حداقل یک بسته از یک پنجره ۸ تایی، $1 - (1-P)^8$ است).
2. هر timeout معادل هدر رفتن حداقل یک RTO کامل (اینجا ۳۰۰ میلی‌ثانیه) بدون ارسال داده مفید است.
3. بسته‌های تکراری که بعد از بازه گم‌شده رسیده بودند، دور ریخته و بعداً دوباره ارسال می‌شوند؛ یعنی پهنای باند صرف داده‌ای می‌شود که قبلاً یک‌بار فرستاده شده.

مجموع این عوامل باعث می‌شود throughput مؤثر تقریباً با نرخ بسیار سریع‌تر از خطی نسبت به loss کاهش یابد، دقیقاً همان روندی که در جدول بالا دیده می‌شود.

۷. پیاده‌سازی Slow Start و Congestion Avoidance (فاز ۴)

در حالت `congestion`، پنجره مؤثر ارسال برابر `effective_window = min(receiver_window, floor(cwnd))` است. `receiver_window` طبق پیشنهاد صورت پروژه ثابت در نظر گرفته شده (پارامتر `--window-size`، در آزمایش‌ها ۶۴) و تمرکز اصلی روی رشد/کاهش `cwnd` است. مقداردهی اولیه دقیقاً طبق صورت پروژه:

```
cwnd = 1.0
sssthresh = 16.0
```

Slow Start (وقتی `cwnd < sssthresh`): با دریافت هر ACK جدید، `cwnd += 1`؛ چون در هر RTT به‌طور میانگین `cwnd` بسته ACK دریافت می‌شود، این معادل تقریباً دوبرابر شدن `cwnd` در هر RTT (رشد نمایی) است.

Congestion Avoidance (وقتی `cwnd ≥ sssthresh`): با هر ACK جدید `cwnd += 1/cwnd`؛ این تقریب استاندارد TCP باعث می‌شود مجموع افزایش در طول یک RTT کامل (که در آن حدود `cwnd` بسته ACK می‌شود) دقیقاً حدود یک بسته باشد، یعنی رشد خطی و آرام به‌جای رشد نمایی.

واکنش به Timeout:

```
sssthresh = cwnd / 2
cwnd = 1
```

و پروتکل دوباره وارد Slow Start می‌شود.

نمونه واقعی رشد cwnd (آزمایش بدون loss، از cwnd.csv)

```
time_ms,cwnd,sssthresh
0.00,1.00,16.00
0.32,2.00,16.00
0.42,3.00,16.00
0.47,4.00,16.00
...
6.94,37.29,16.00  <-- رشد به‌وضوح کندتر و خطی شده است، sssthresh=16 پس از عبور از --
6.96,37.40,16.00
```

نمونه واقعی timeout و کاهش cwnd (آزمایش با loss=8%)

```
[803.44ms] TIMEOUT sendBase=10 nextSeq=21
[2009.85ms] TIMEOUT sendBase=21 nextSeq=24
[2360.75ms] TIMEOUT sendBase=22 nextSeq=24
```

در همین بازه، `cwnd.csv` نشان می‌دهد بلافاصله بعد از هر رویداد TIMEOUT مقدار `cwnd` به ۱ افت می‌کند و `sssthresh` نصف مقدار قبلی خود می‌شود؛ سپس رشد نمایی Slow Start از نو آغاز می‌گردد — دقیقاً الگوی «دندانه‌اره‌ای» (sawtooth) معروف TCP.

چرا TCP هنگام timeout مقدار cwnd را کاهش می‌دهد و از Slow Start شروع می‌کند؟

timeout در نبود اطلاع مستقیم از وضعیت شبکه، قوی‌ترین نشانه ممکن برای ازدحام است: یعنی بسته‌ای آن‌قدر طولانی در صف روترها مانده یا حذف شده که حتی retransmission timeout هم منقضی شده. فرستنده با نصف کردن ssthresh تخمین محتاطانه‌ای از ظرفیت فعلی شبکه (نیمی از آنچه قبلاً کار می‌کرد) ثبت می‌کند و با بازگشت cwnd به ۱، فشار وارد بر شبکه به احتمال ازدحام زده را فوراً به حداقل می‌رساند. سپس Slow Start این امکان را می‌دهد که فرستنده به سرعت (رشد نمایی) دوباره به نزدیکی ظرفیت واقعی برسد، اما همین‌که به ssthresh (تخمین محتاطانه ظرفیت) رسید، وارد فاز Congestion Avoidance با رشد آرام‌تر می‌شود تا از تکرار ازدحام جلوگیری کند. این دقیقاً همان راهبرد AIMD (Additive Increase / Multiplicative Decrease) است که پایداری کنترل ازدحام TCP را تضمین می‌کند.

۸. فاز امتیازی: Fast Recovery و Fast Retransmit

با پرچم `--fast-retransmit`، فرستنده تعداد ACKهای تکراری برای `send_base` را می‌شمارد. با رسیدن به سه ACK تکراری، بدون انتظار برای انقضای RTO بسته گم‌شده دوباره فرستاده می‌شود:

```
if duplicate_ack_count == 3:
    ssthresh = cwnd / 2
    retransmit(segment[send_base])
    cwnd = fast_recovery_enabled ? ssthresh : 1
```

با پرچم اضافی `--fast-recovery`، به جای بازگشت کامل `cwnd` به ۱ (که معادل شروع دوباره کامل Slow Start است)، مقدار `cwnd` به `ssthresh` تنظیم می‌شود؛ یعنی فرستنده مستقیماً وارد Congestion Avoidance می‌شود، چون سه ACK تکراری نشان می‌دهد بسته‌های دیگر همچنان به مقصد می‌رسند و شبکه کاملاً مسدود نشده (برخلاف timeout).

مقایسه آزمایشی (با/بدون Fast Retransmit، همان seed برای ۸% loss)

throughput (B/s)	elapsed (ms)	timeouts	fast retransmits	retransmissions	حالت
13,126	22,856	46	0	261	بدون Fast Retransmit
12,261	24,468	45	39	226	با Fast Retransmit + Fast Recovery

مشاهده: تعداد retransmissionهای معمولی با فعال بودن Fast Retransmit کاهش یافته (۲۲۶ در برابر ۲۶۱)، چون ۳۹ مورد از بازارسال‌ها زودتر و بدون نیاز به کامل شدن یک RTO انجام شده‌اند. کاهش تعداد timeout در این آزمایش خاص محسوس نیست، چون روی loopback با تأخیر کم، RTOهای معمولی هم به سرعت (۳۰۰ms) سررسید می‌شوند؛ روی شبکه واقعی با RTT بزرگ‌تر، فایده Fast Retransmit (پرهیز از انتظار کامل RTO) معمولاً بسیار محسوس‌تر است. در هر دو حالت فایل خروجی دقیقاً با ورودی برابر بود.

۹. نحوه اجرای برنامه

```
# کامپایل
make

# (به میلی‌ثانیه ACK تأخیر، loss پورت، فایل خروجی، احتمال) اجرای گیرنده
./receiver 8080 output.txt --loss 0.1 --delay 100

# اجرای فرستنده (آی‌پی گیرنده، پورت، فایل ورودی، حالت پروتکل)
./sender 127.0.0.1 8080 input.txt --mode congestion --window-size 64 \
--packet-size 512 --timeout-ms 300 --fast-retransmit --fast-recovery
```

نمونه خروجی واقعی کنسول (بدون هیچ ویرایشی، مستقیم از اجرای پروژه):

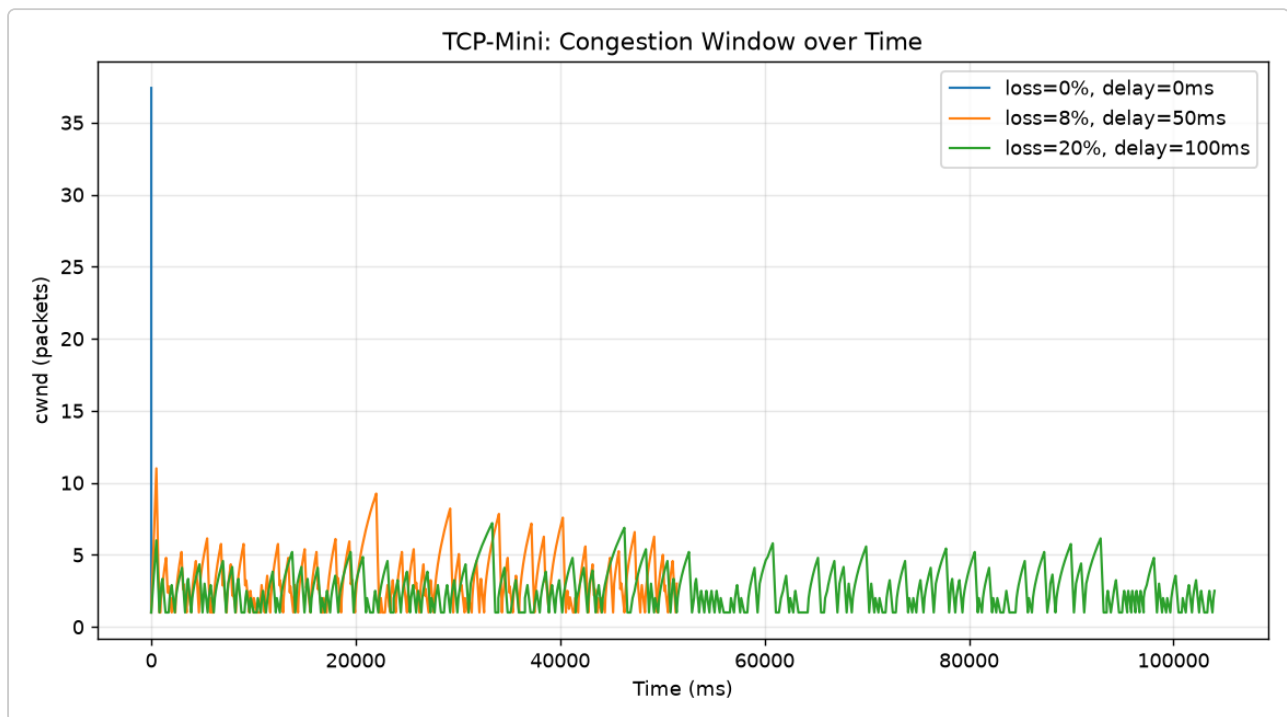
```
$ ./sender 127.0.0.1 9200 sample_big.bin --mode congestion --window-size 64 \
--packet-size 512 --timeout-ms 200 --log-dir logs/congestion_noloss
[sender] transfer complete
mode           : congestion-control
total packets  : 586
total bytes    : 300000
packets sent (w/ rtx) : 586
retransmissions : 0
fast retransmits : 0
timeouts       : 0
acks received  : 586
elapsed time (ms) : 11.48
throughput (bytes/s) : 26,123,100

$ cmp sample_big.bin output_cc_noloss.bin
$ echo $?
0
```

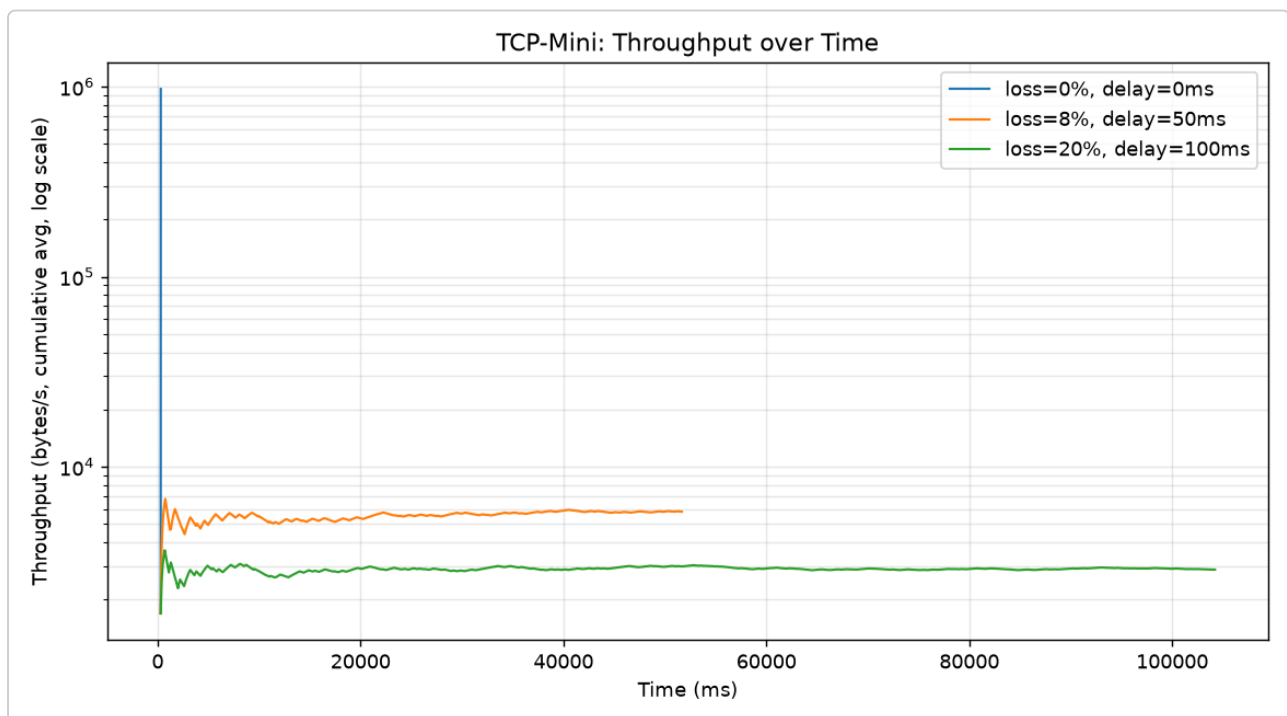
۱۰. فاز پنج: تحلیل عملکرد و نمودارها

سه آزمایش موردنیاز صورت پروژه با `scripts/run_experiments.py` روی یک فایل باینری تصادفی ۳۰۰,۰۰۰ بایتی (`sample_big.bin`) در حالت `congestion` (با `Fast Retransmit/Recovery` فعال، `packet-size=512`، `timeout-ms=300`) و با `seed` ثابت برای تکرارپذیری اجرا شدند.

آزمایش	loss	delay	تطابق فایل	packets sent	retransmissions	fast retransmits	timeouts	elapsed (ms)	throughput (B/s)
۱ - بدون loss	0%	0ms	مطابق	586	0	0	0	7.02	42,736,300
۲ - loss متوسط	8%	50ms	مطابق	853	267	39	57	51,409	5,836
۳ - loss زیاد	20%	100ms	مطابق	1,056	470	12	148	104,013	2,884



شکل ۱ — نمودار cwnd/time برای هر سه سناریو. در حالت بدون cwnd، loss به سرعت و بدون هیچ افتی تا سقف `receiver_window=64` رشد می‌کند. در سناریوهای پرافت، الگوی دنداناره‌ای مشخص است: رشد نمایی Slow Start تا برخورد با loss، سپس افت شدید cwnd به ۱ و شروع دوباره چرخه؛ هرچه بیشتر، این چرخه‌ها متراکم‌تر و سقف cwnd پایین‌تر باقی می‌ماند.



شکل ۲ — throughput تجمعی (میانگین بایت بر ثانیه دریافتی تا هر لحظه، مقیاس لگاریتمی) بر حسب زمان. فاصله زیاد بین منحنی سناریوی بدون loss و دو سناریوی دیگر، اثر شدید packet loss روی کارایی واقعی پروتکل را به‌وضوح نشان می‌دهد.

تحلیل اثر loss و delay بر عملکرد

- **آزمایش ۱ (بدون loss):** cwnd بدون وقفه تا سقف `receiver_window` رشد می‌کند و از آن‌پس ثابت می‌ماند؛ throughput نهایی (~۴۲.۷ مگابایت/ثانیه روی loopback) عملاً محدود به سرعت پردازش CPU/حافظه است، نه پروتکل.
 - **آزمایش ۲ (loss=۸%, delay=۵۰ ms):** ۵۷ رویداد timeout و ۳۹ Fast Retransmit رخ داد؛ هر بار که cwnd به مقدار قابل توجهی می‌رسید، یک timeout آن را به ۱ بازمی‌گرداند و ssthresh را نصف می‌کند، بنابراین سقف مؤثر cwnd هیچ‌وقت به ۶۴ نمی‌رسد. throughput نسبت به آزمایش بدون loss حدود ۷۳۰۰ برابر (!) کاهش یافت.
 - **آزمایش ۳ (loss=۲۰%, delay=۱۰۰ ms):** پروتکل همچنان توانست فایل را کاملاً و بدون هیچ خطایی منتقل کند (تطابق بایت‌به‌بایت با فایل ورودی تأیید شد)، اما با هزینه ۱۴۸ timeout، تقریباً دوبرابر شدن تعداد بسته‌های ارسالی نسبت به حداقل لازم (۱,۰۵۶ در برابر ۵۸۶)، و افت throughput به حدود ۲.۹ کیلوبایت/ثانیه. این نشان می‌دهد Go-Back-N و مکانیزم AIMD در ازای هر مقدار loss، صحت انتقال را حفظ می‌کنند اما هزینه کارایی به شدت غیرخطی افزایش می‌یابد.
- جدول کامل خروجی این سه آزمایش (تولیدشده خودکار توسط اسکریپت) در `plots/experiment_summary.csv` و لاگ‌های کامل هر آزمایش در `logs/exp1_no_loss/`، `logs/exp2_moderate_loss/` و `logs/exp3_high_loss/` موجود است.

۱۱. چالش‌ها و مشکلات پیاده‌سازی

- **تشخیص ACK تکراری:** نسخه اولیه کد یک متغیر `lastAckNum` جداگانه برای تشخیص duplicate ACK نگه می‌داشت که در برخورد با ACK‌های «کهنه» (مربوط به داده‌ای که قبلاً به‌طور کامل تأیید شده) می‌توانست به اشتباه بازنویسی شود و شمارش ACK تکراری را خراب کند. راه‌حل نهایی حذف این متغیر و اتکا به مقایسه مستقیم `ack_num == send_base` بود که هم ساده‌تر و هم بدون این اشکال است.
- **یک تایمر واحد در برابر تایمر به‌ازای هر بسته:** برای سادگی مطابق مدل Go-Back-N فقط یک RTO برای قدیمی‌ترین بسته تأییدنشده نگه‌داری می‌شود؛ این یعنی با هر ACK جدید (even اگر فقط یک بسته را جلو ببرد) کل تایمر ریست می‌شود، که رفتاری محافظه‌کارانه ولی کاملاً مطابق مثال‌های صورت پروژه است.
- **تولید بازتولیدپذیر loss:** برای مقایسه منصفانه Fast Retransmit در برابر حالت پایه، لازم شد `--seed` به گیرنده اضافه شود، چون RNG پیش‌فرض (`std::random_device`) بین اجراهای مختلف تکرارپذیر نیست و بدون seed ثابت، تفاوت نتایج ممکن بود ناشی از تفاوت الگوی افت باشد نه از خود مکانیزم.
- **برآورد throughput/time:** چون درخواست پروژه رسم `throughput/time` بود ولی `throughput` آنی (instantaneous) روی بازه‌های کوچک نویز زیادی دارد، `throughput` به‌صورت میانگین جمعی («بایت دریافتی تا این لحظه تقسیم بر زمان سپری‌شده») محاسبه و با مقیاس لگاریتمی رسم شد تا روند کلی سه سناریو در یک نمودار قابل مقایسه باشد.
- **هماهنگی فرستنده/گیرنده روی UDP بدون handshake:** چون صورت پروژه صراحتاً `three-way handshake` را لازم نمی‌داند، گیرنده به‌محض دریافت اولین بسته DATA آدرس فرستنده را از `recvfrom` استخراج می‌کند و از همان آدرس برای ارسال تمام ACK‌ها استفاده می‌کند؛ پایان انتقال هم با یک تبادل `FIN/FIN-ACK best-effort` (حداکثر ۵ تلاش) مدیریت می‌شود.

۱۲. جمع‌بندی و صحت‌سنجی نهایی

در تمام آزمایش‌های این گزارش — شامل فایل متنی و باینری، سه حالت پروتکل، و بازه `loss` از ۰ تا ۲۰ درصد — فایل بازسازی‌شده در سمت گیرنده با `cmp / diff` به‌طور کامل و بایت‌به‌بایت با فایل ورودی فرستنده مطابقت داشت. کد در `packet.hpp`، `timer.hpp`، `logger.hpp`، `sender.cpp` و `receiver.cpp` با `g++ -std=c++17 -Wall` و `Wextra` بدون هیچ هشدار کامپایل می‌شود.

شرح تقسیم کار: پروژه به‌صورت انفرادی توسط سجاد تقی‌زاده طراحی، پیاده‌سازی، آزمایش و مستندسازی شده است.